



Pipeline Hazards and Instruction Scheduling

In my last two articles, I had explained the concept of pipelining and pipeline hazards. We saw how the latter reduce the ideal speed-up gained by pipelining. Here, we will look at how the pipeline feature can be utilised by exploiting Instruction Scheduling.

Most processors overcome pipeline hazards (data, structure and control) by inserting pipeline stalls, by data forwarding, or by a combination of both. Stalls decrease the overall performance of the system, since they cause a delay in execution. Since the throughput of the instruction pipeline is decreased by stalls, we need to find out ways to minimise stalls—by rescheduling instructions such that the hazardous instruction combination does not occur.

Instruction scheduling to minimise stalls

To understand this, consider the following instructions:

```
ADD R1, R2, R3 //Instruction 10
MUL R4, R1, R2 //Instruction 11
MOV R5, R6    //Instruction 12
```



Note: In the above example, let us assume that the first operand is the destination operand, and the remaining are the source operands.

As we see in the above example, the *ADD* instruction updates the value of register *R1*, and the *MUL* instruction uses the value of *R1* as updated by the *ADD* instruction. The *ADD* instruction will update the *R1* register in the WB stage, but the *MUL* instruction will read